Marissa Ratschki

CMPT-308

LAB 04

TOPOLOGY:

SYSTEM LAYER
[macOS Tahoe 26.0.1 (Host Machine)]
↓
[pgAdmin (Client GUI Application)]
↓
[PostgreSQL 18 (Database Server/DBMS)]
↓

DATA LAYER
[Database: cmpt308-lab04]
↓
[Schema: public]
↓
[Tables: Students, Courses, Enrollments]

PART A:

Query Query History

```sql
SELECT s.name, e.course_id
FROM Students s
INNER JOIN Enrollments e
ON s.student_id = e.student_id
WHERE e.term = '2026SP';
```

Data Output Messages Notifications

Showing row

	name character varying (100)	course_id character varying (10)
1	Mila	CMPT308
2	John	CMPT308
3	Julia	CMPT307
4	Artyom	CMPT308
5	Lucy	CMPT423
6	Tyler	CMPT307

Figure 1: Part a1

Query Query History

```sql
SELECT s.name, c.title, e.term
FROM Students s
INNER JOIN Enrollments e
ON s.student_id = e.student_id
INNER JOIN Courses c
ON e.course_id = c.course_id
WHERE e.term = '2026SP';
```

Data Output Messages Notifications

Showing rows: 1 to 6

	name character varying (100)	title character varying (100)	term character varying (10)
1	Artyom	Database Management	2026SP
2	John	Database Management	2026SP
3	Mila	Database Management	2026SP
4	Tyler	Internetworking	2026SP
5	Julia	Internetworking	2026SP
6	Lucy	Hacking and Pentesting	2026SP

Figure 2: Part a2

	student_id [PK] integer	name character varying (100)	major character varying (50)	class_year integer
1	29	Mila	CS	2026
2	57	John	CS	2027
3	48	Julia	DS	2026
4	25	Artyom	CS	2026
5	34	Lucy	DS	2028
6	55	Tyler	CS	2026

Query Query History

```sql
SELECT c.course_id, c.title, COUNT(e.student_id) AS enrollments
FROM Courses c
LEFT JOIN Enrollments e
ON c.course_id = e.course_id
AND e.term = '2026SP'
GROUP BY c.course_id, c.title
ORDER BY c.course_id;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1

	course_id [PK] character varying (10)	title character varying (100)	enrollments bigint
1	CMPT307	Internetworking	2
2	CMPT308	Database Management	3
3	CMPT423	Hacking and Pentesting	1
4	CMPT450	Independent Study	0
5	CMPT466	Special Topics	0

Figure 3: Part a3

PART B:

Query Query History

```sql
SELECT major, COUNT(*) AS num_students
FROM Students
GROUP BY major
ORDER BY num_students DESC;
```

Data Output Messages Notifications

	major character varying (50)	num_students bigint
1	CS	4
2	DS	2

Figure 4: Part b1

Query Query History

```sql
SELECT c.course_id, c.title, COUNT(*) AS num_enrollments
FROM Courses c
JOIN Enrollments e
ON c.course_id = e.course_id
WHERE e.term = '2026SP'
GROUP BY c.course_id, c.title;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1

	course_id [PK] character varying (10)	title character varying (100)	num_enrollments bigint
1	CMPT307	Internetworking	2
2	CMPT308	Database Management	3
3	CMPT423	Hacking and Pentesting	1

Figure 5: Part b2

Query Query History

```sql
SELECT course_id, COUNT(*) AS num_enrollments
FROM Enrollments
WHERE term = '2026SP'
GROUP BY course_id
HAVING COUNT(*) >= 3;
```

Data Output Messages Notifications

Showing

	course_id character varying (10)	num_enrollments bigint
1	CMPT308	3

Figure 6: Part b3

PART C:

Query Query History

```sql
SELECT name
FROM Students
WHERE student_id IN (
    SELECT student_id
    FROM Enrollments
    WHERE course_id = 'CMPT308'
    AND term = '2026SP'
);
```

Data Output Messages Notifications

	name character varying (100)
1	Artyom
2	Mila
3	John

Figure 7: Part c1

Query Query History

```sql
SELECT course_id, title
FROM Courses c
WHERE EXISTS (
    SELECT *
    FROM Enrollments e
    WHERE e.course_id = c.course_id
    AND e.term = '2026SP'
);
```

Data Output Messages Notifications

Showing ro

	course_id [PK] character varying (10)	title character varying (100)
1	CMPT308	Database Management
2	CMPT307	Internetworking
3	CMPT423	Hacking and Pentesting

Figure 8: Part c2

Query Query History

```sql
SELECT s.name
FROM Students s
JOIN Enrollments e
ON s.student_id = e.student_id
WHERE e.term = '2026SP'
GROUP BY s.student_id, s.name
HAVING COUNT(*) >= 2;
```

Data Output Messages Notifications

name character varying (100)

Figure 9: Part c3

PART D:

Query Query History

```sql
SELECT student_id, name
FROM Students
WHERE major = 'CS'

UNION

SELECT student_id, name
FROM Students
WHERE major = 'DS';
```

Data Output Messages Notifications

	student_id integer	name character varying (100)
1	25	Artyom
2	29	Mila
3	34	Lucy
4	48	Julia
5	55	Tyler
6	57	John

Figure 10: Part d1

Query Query History

```sql
SELECT student_id, name
FROM Students
WHERE major = 'CS'

EXCEPT

SELECT s.student_id, s.name
FROM Students s
JOIN Enrollments e
ON s.student_id = e.student_id
WHERE e.course_id = 'CMPT308'
AND e.term = '2026SP';
```

Data Output Messages Notifications

	student_id integer	name character varying (100)
1	55	Tyler

Figure 11: Part d2

PART E: AI PROMPTING:

CHATGPT:

```sql
SELECT c.course_id, c.title,
       COUNT(DISTINCT e.student_id) AS num_students
FROM Courses c
LEFT JOIN Enrollments e
    ON c.course_id = e.course_id
    AND e.term = '2026SP'
GROUP BY c.course_id, c.title
ORDER BY num_students DESC, c.course_id;
```

COPILOT:

```sql
SELECT
    c.course_id,
    c.title,
    COUNT(DISTINCT e.student_id) AS student_count
FROM courses c
LEFT JOIN enrollments e
    ON c.course_id = e.course_id
    AND e.term = '2026SP'
GROUP BY
    c.course_id,
    c.title
ORDER BY
    student_count DESC,
    c.course_id;
```

ANALYSIS:

For purposes of this lab, Copilot and ChatGPT were chosen to provide an SQL query based on the following prompt: *"Write a SQL query that returns a list of course_id and title for all courses, and also returns the number of distinct students enrolled in each course for term '2026SP'. Include courses with zero enrollments. Sort by the count (highest to lowest), then by course_id. Use the tables Students(student_id,...), Courses(course_id,...), Enrollments(student_id, course_id, term, ...)."* ChatGPT's answer, as detailed in the above figure, is correct. The query, when run in PgAdmin retrieves all courses, with their number of distinct enrollments, sorted by highest to lowest enrollment numbers, then by course ID. In

addition, ChatGPT correctly included the two courses in the dataset with 0 enrollments. The query provided by Copilot was also correct, providing the same result. Although each model resulted in the same outcome, each on the first prompt, ChatGPT is preferable. This is due to its more streamlined and familiar syntax, where each clause is provided on the same line. Copilot's syntax differed in that individual clauses were broken up into different lines, straying from a familiar style of syntax where each clause is typically presented on a single line.